Compatible Microservice Rollbacks

Day 04 | KUBERNETES / EKS / AKS / GKE

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

A stable release is a compatible set of components, not a collection of individually healthy images. During a rolling transition, mixed versions can coexist. The rollback sequence must therefore keep supported caller and provider contracts intact.

The Scenario Service A v2 calls B v2. B fails, but A v2 cannot use B v1.

The Mistake Rolling back B alone creates an incompatible combination.

The Target Restore a tested version pair, not two unrelated versions labelled stable.

Before you start

Immutable image versions or digests; a tested release record; API contract knowledge; access to current schema and message-format changes; an end-to-end test.

Diagnostic checks

```
kubectl rollout history deployment/service-a
kubectl rollout history deployment/service-b

# Check API errors and release records
```

Map The Dependency: Which API contract does each version require? Can mixed versions serve traffic?

Check State: Confirm old code can use the current database schema and message format.

Apply the cause specific fix

Fix The Confirmed Cause

```
# Release record, NOT Kubernetes API YAML
release: stable-2026-10-01
services:
  service-a: v1.8.0
  service-b: v2.3.0
validation:
  - contract-tests
  - checkout-end-to-end
```

Coordinate In Ci/Cd: Redeploy this tested pair in a compatible order. If no mixed state is safe, pause traffic or switch a complete stack.

Do Not Assume: A must not always roll back first, and B must not always roll back first. Use the compatibility evidence.

Validate the result

```
kubectl rollout status deployment/service-a
kubectl rollout status deployment/service-b

# Execute the real end-to-end test
```

Expected: Both services are ready and the business transaction succeeds.

Gitops?: Update the source of truth so reconciliation does not reapply the failed version.

Practice exercise

In a lab, record one compatible pair, introduce a deliberate API mismatch and exercise the recovery sequence. Observe which mixed state is safe before choosing which service moves first.

Implementation notes

Release YAML is an example input format for your pipeline; Kubernetes does not interpret it. Deployment rollback restores Pod template revisions, not business data or all external configuration. Helm releases must be coordinated if separate. A live failure unrelated to code may not need rollback. Keep immutable image digests and known-compatible release manifests.

Interview question

If B is failing, must A roll back? Only when the target B version is incompatible with the current A, or release evidence otherwise requires a coordinated recovery.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1